

ME2605 / DDA6204 · Neural Net Training · Fall 2026

Lecture 2: Backpropagation and Signal Propagation

Gradients on the computation graph, and the energy-preserving initialization

Ruoyu Sun

Today's Plan

Part 1 — Backpropagation — from two matrices to the linear network, and the FCN in matrix form

Part 2 — Signal Propagation I — explore what makes it train; then the principle and its proof

Learning Goals

By the end of this lecture you should be able to:

- Write the FCN in both forms, and name h (pre-activation) and z (post-activation).
- Write the gradient of an FCN in matrix form, and explain why $\partial F / \partial W_l$ has rank at most 1.
- State the energy-preserving principle and use it to choose the variance of the weights.
- Understand what the derivation really needs — a zero mean and a fixed variance, not a Gaussian.
- Know the roles of classical initializations — Bottou–LeCun, Xavier, Kaiming — in practice.

PART 1

Backpropagation

The computation graph, one layer at a time

The Network Expression

We introduce a fully connected network.

$$z_0 = x,$$

$$h_l = W_l z_{l-1} + b_l, \quad l = 1, \dots, L, \quad (\text{pre-activation}),$$

$$z_l = \varphi(h_l), \quad l = 1, \dots, L-1, \quad (\text{post-activation}),$$

$$f_{\theta}(x) = h_L,$$

$$\theta = \{W_l, b_l\}_{l=1}^L, \quad W_l \text{ is } d_l \times d_{l-1}.$$

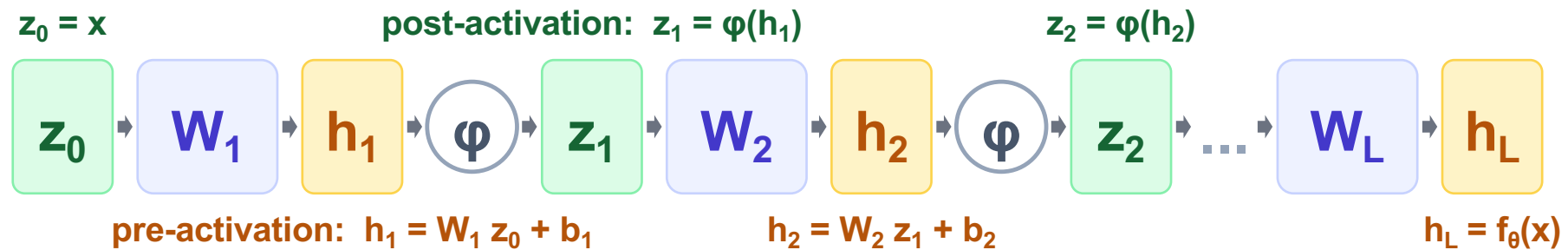
W_l has d_l rows and d_{l-1} columns: it takes z_{l-1} to h_l .

Illustration of FCN

Pay attention to **h** and **z**:

h (pre-activation): the one before the nonlinear activation

z (post-activation): the one after the nonlinear activation

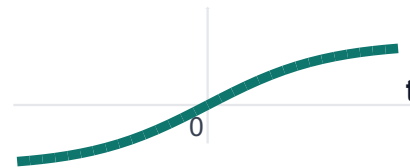


The Nonlinear Activation Works Coordinate by Coordinate

ReLU(t)



tanh(t)



$$\varphi(h_1, \dots, h_d) = (\varphi(h_1), \dots, \varphi(h_d)),$$

$$\text{e.g. } \text{ReLU}(t) = \max\{t, 0\}, \quad \tanh(t) = \frac{e^t - e^{-t}}{e^t + e^{-t}},$$

$$\text{ReLU}(h_1, h_2) = (\max\{h_1, 0\}, \max\{h_2, 0\}).$$

$$\text{ReLU}(1.5, -0.7) = (1.5, 0); \tanh(1.5, -0.7) \approx (0.905, -0.604).$$

Nested Form, and the Supervised Loss

Unrolling the recursion gives a nest; the objective averages a loss over the data.

$$f_{\theta}(x) = W_L \varphi(\cdots \varphi(W_1 x + b_1) \cdots) + b_L,$$

Sometimes,
we use this form:

$$f_{\theta}(x) = W_L \varphi(\cdots \varphi(W_1 x) \cdots) \quad (\text{no-bias form}),$$

Empirical loss
(objective function):

$$\min_{\theta} F(\theta) \triangleq \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)).$$

ℓ is the per-sample loss:

e.g. $\|y - f\|^2$ for regression,

$\log(1+e^{-yz})$ for binary classification,

cross entropy function for multi-class classification

SGD

The objective is a sum over samples, so one step may use one sample — or several.

$$\min_{\theta} F(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i(\theta), \quad \ell_i(\theta) \triangleq \ell(y_i, f_{\theta}(x_i)),$$

SGD: draw i_t at random, then $\theta_{t+1} = \theta_t - \eta \nabla \ell_{i_t}(\theta_t)$,

minibatch B_t of size B : $\theta_{t+1} = \theta_t - \frac{\eta}{B} \sum_{i \in B_t} \nabla \ell_i(\theta_t)$.

$B = 1$ is SGD, $B = n$ is full gradient descent — batch size sits between them.

Gradient: Simple Case of Two-Layer Linear Nets

The gradient from a simple matrix factorization — a special case of a two-layer linear net.

$$F = \left\| \underset{m \times n}{M} - \underset{m \times k}{W_1} \underset{k \times n}{W_2} \right\|_F^2, \quad \Delta \triangleq M - W_1 W_2,$$

$$\frac{\partial F}{\partial W_1} = -2 \Delta W_2^\top, \quad \frac{\partial F}{\partial W_2} = -2 W_1^\top \Delta.$$

- Each gradient is built from the other matrix, transposed.
- Dimension matching: matrix multiplication shall have dimension match.

One Matrix, With Something on Each Side

A warm-up before the general network.

$$F(W) = \left\| \underset{p \times q}{Y} - \underset{p \times m}{A} \underset{m \times n}{W} \underset{n \times q}{B} \right\|_F^2,$$

$$\frac{\partial F}{\partial W} = -2 \underset{m \times p}{A}^\top \underset{p \times q}{\Delta} \underset{q \times n}{B}^\top.$$

- The left neighbour comes back transposed on the left; the right neighbour, on the right.

A and B are placeholders — in real network, they will be specified.

Deep Linear Network

Next, we consider deep linear network.

$$z_l = W_l z_{l-1}, \quad z_0 = x,$$

$$z_L = W_L W_{L-1} \cdots W_1 x, \quad F = \frac{1}{2} \|z_L - y\|^2.$$

The Gradient of Deep Linear Network

The gradient of this loss (for deep linear net) is:

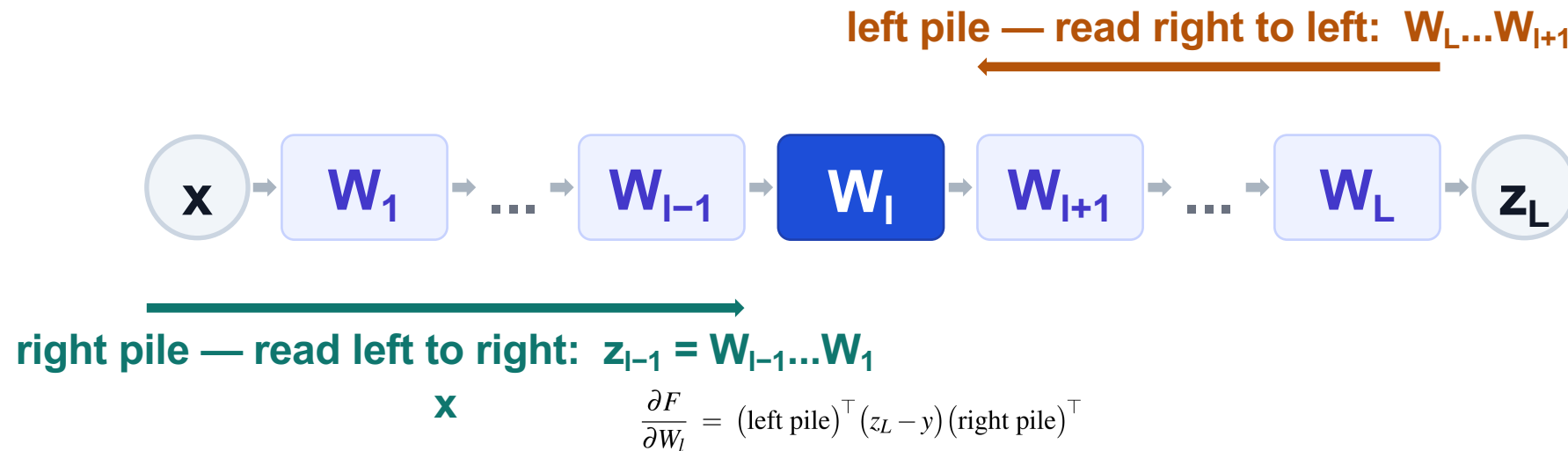
$$\frac{\partial F}{\partial W_l} = \underbrace{\left(W_L \cdots W_{l+1}\right)^\top}_{\text{left pile}} \underbrace{\left(z_L - y\right)}_{\text{residual}} \underbrace{\left(z_{l-1}\right)^\top}_{\text{right pile}}$$

- A pile of W's on the left, a pile of W's on the right.

Use the rule with $A = W_L \cdots W_{l+1}$, $B = z_{l-1}$ and $Y = y$.

Forward and Backward

Both piles are products of weights.



Forward pass: builds the right pile;

Backward pass: builds the left pile.

Name the Two Piles: the Error and the Layer Output

Now, we name the two parts in the gradient: backward error and layer output.

$$\frac{\partial F}{\partial W_l} = e_l (z_{l-1})^\top,$$

$$e_L \triangleq z_L - y, \quad e_l = (W_{l+1})^\top e_{l+1}.$$

Nonlinear Network

Finally, we are ready to see the gradient of a nonlinear network.

$$h_l = W_l z_{l-1} + b_l, \quad z_l = \varphi(h_l),$$
$$\frac{\partial F}{\partial W_l} = e_l (z_{l-1})^\top \quad \text{still — but with } e_l = ?$$

Same two piles. Only the backward one changes.

- The forward pile is similar: z_{l-1} is whatever the forward pass produced.
- The backward pile is where φ enters, what is it?

Gradient: Simple Case 1 — One Scalar

The smallest nonlinear case: one weight, one input, one scalar output.

$$z = \varphi(wx), \quad h_0 = w_0x, \quad \left. \frac{\partial z}{\partial w} \right|_{w=w_0} = \varphi'(w_0x)x = \varphi'(h_0)x,$$

$$\text{linearize about } h_0 : \quad \varphi(h) \approx \varphi'(h_0)h + \text{const} \implies z \approx \varphi'(h_0)wx \implies \left. \frac{\partial z}{\partial w} \right|_{w=w_0} = \varphi'(h_0)x,$$

$$\text{general } h, w : \quad z = \varphi(h)wx \implies \frac{\partial z}{\partial w} = \varphi(h)x.$$

Lesson: linearize about the current point — the derivative is unchanged.

Gradient: Simple Case 2 — the Matrix Version

Now W is a matrix and x, z are vectors.

$$z = \varphi(Wx), \quad h = Wx, \quad \frac{\partial z}{\partial x} = DW, \quad \text{which is } d_{\text{out}} \times d_{\text{in}},$$

$$\varphi(Wx) \approx \text{diag}(\varphi'(h)) Wx = \begin{matrix} & & & & x \\ & & & & d_{\text{in}} \times 1 \\ D & W \\ d_{\text{out}} \times d_{\text{out}} & d_{\text{out}} \times d_{\text{in}} \end{matrix},$$

$$D = \text{diag}(\varphi'(h_1), \dots, \varphi'(h_{d_{\text{out}}})) .$$

Component i : $z_i \approx \varphi'(h_i) \cdot (Wx)_i$.

The matrix version of φ' is the diagonal D — one derivative per coordinate.

The Linearization Trick

Apply the rule to every layer: define a diagonal matrix D_l

$$D_l = \text{diag}(\varphi'(h_{l,1}), \dots, \varphi'(h_{l,d_l})), \quad h_l = W_l z_{l-1} + b_l.$$

- D_l is set by the current h_l , so it changes as training moves.
- For nonlinear net, the backpropagated error is:

$$\mathbf{e}_l = \mathbf{D}_l (\mathbf{W}_{l+1})^\top \mathbf{e}_{l+1}$$

or

$$e_l = \left(W_L D_{L-1} \cdots W_{l+2} D_{l+1} W_{l+1} D_l \right)^\top e,$$

- The gradient is:

$$\frac{\partial F}{\partial W_l} = e_l (z_{l-1})^\top$$

If interested in a rigorous derivation, check my book draft (share later)

D by Hand: One Layer, Two Units

Same two units, tanh, and every number written out.

$$W_1 = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix}, \quad z_0 = x = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$h_1 = W_1 x = \begin{bmatrix} 2 \cdot 1 - 1 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

$$z_1 = \tanh(h_1) = \begin{bmatrix} 0.7616 \\ 0.9640 \end{bmatrix},$$

$$D_1 = \text{diag}(1 - z_1[1]^2, 1 - z_1[2]^2) = \text{diag}(0.4200, 0.0707).$$

For tanh, $\phi'(h[i]) = 1 - z[i]^2$, so nothing new to compute.

D by Hand: the Way Back

Now put that D to work on an error arriving from the layer above.

$$W_2 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad e_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$(W_2)^\top e_2 = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix},$$

$$e_1 = D_1 \odot (W_2)^\top e_2 = \begin{bmatrix} 0.4200 \times 2 \\ 0.0707 \times 1 \end{bmatrix} = \begin{bmatrix} 0.8400 \\ 0.0707 \end{bmatrix},$$

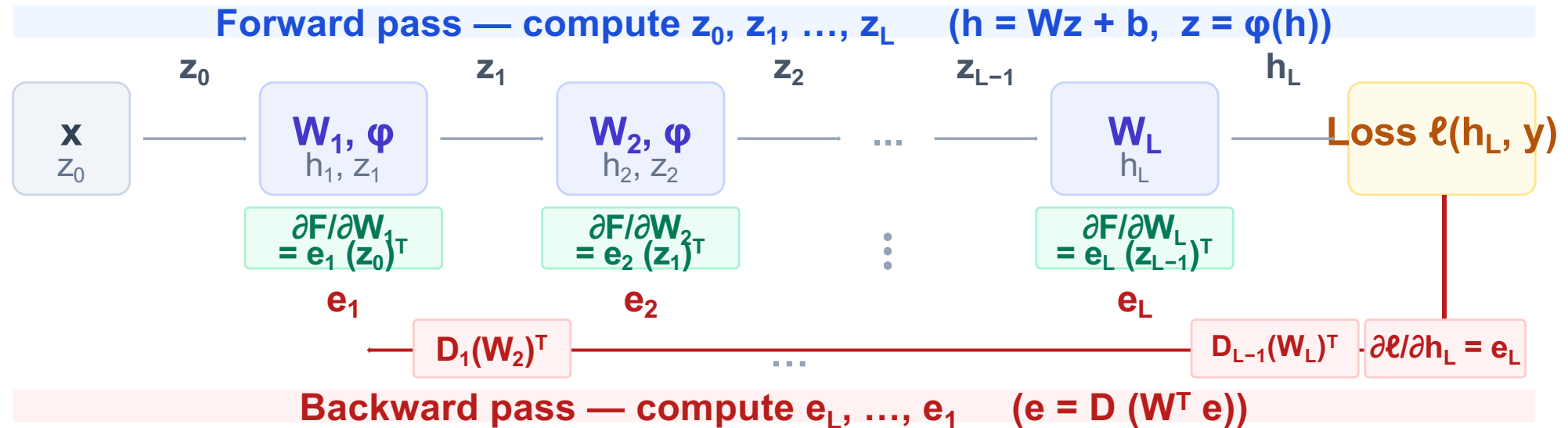
$$\frac{\partial F}{\partial W_1} = e_1 (z_0)^\top = \begin{bmatrix} 0.8400 & 0.8400 \\ 0.0707 & 0.0707 \end{bmatrix}.$$

D never mixes coordinates: two separate multiplications, one per row.

Unit 2 gets a small gradient because $D_1[2]$ is small — D sets the scale.

Back-Propagation on FCN

Illustration of back-propagation on FCN:



Forward: $h_l = W_l z_{l-1} + b_l, z_l = \phi(h_l)$. Backward: $e_l = D_l (W_{l+1})^T e_{l+1}$.

Space and Time for Back-Propagation

Every layer has width d .

Two ways to get all L gradients.

Naïve way: Compute every matrix product in the gradient of each layer, from scratch.

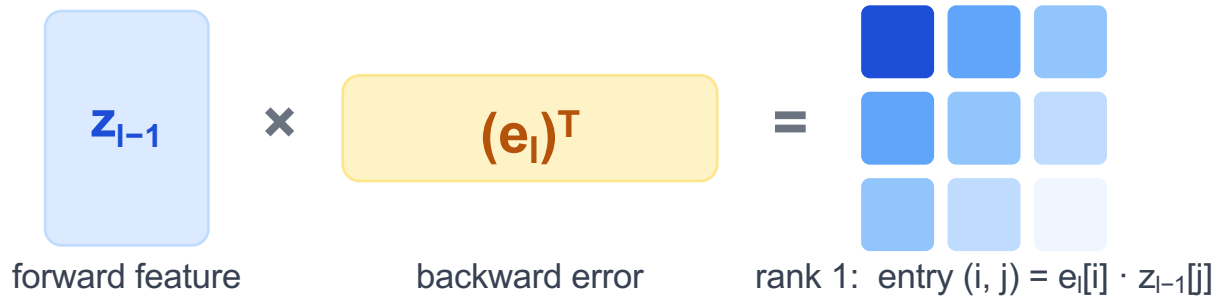
naive $L(L - 1)$ matrix products, $O(d)$ stored,

back-prop $2L$ matrix products, $(L + 1)d$ stored.

Reuse is the major trick:

computing $\mathbf{e}_l = \mathbf{D}_l (\mathbf{W}_{l+1})^T \mathbf{e}_{l+1}$ only requires two extra “matrix times vector” products.

At Most Rank 1 per Sample, at Most B per Minibatch



$$\text{one sample } s : \quad \partial F_s / \partial W_l = e_l (z_{l-1})^T \quad \text{rank} \leq 1,$$

$$\text{minibatch } B_t \text{ of size } B : \quad \partial F_{B_t} / \partial W_l = \frac{1}{B} \sum_{s \in B_t} \partial F_s / \partial W_l \quad \text{rank} \leq B.$$

One sample gives rank at most 1; 64 samples in a minibatch give rank at most 64.

Demo 1 — Backprop on the Computation Graph

Interactive demo, one click per layer: forward, then backward, then the update.

ruoyus.github.io/me2605/demos/backprop-lab/

Phase 1 · forward — $h_l = W_l z_{l-1} + b_l$, $z_l = \phi(h_l)$ — one layer at a time.

Phase 2 · backward — $e_l = D_l (W_{l+1})^T e_{l+1}$, starting from e_L .

Phase 3 · update — $W_l \leftarrow W_l - \eta e_l (z_{l-1})^T$, again one layer at a time.

Quiz 1 — Backpropagation

Ten-minute in-class quiz.

Format — 12 multiple-choice questions, three parallel forms (A / B / C) so neighbours get different papers.

Covered — a rule of matrix function gradient; the linear-network gradient in W 's only; the two piles; the linearization trick

BREAK

Break



10 minutes

PART 2

Signal Propagation I

From a network that will not train to the principle that fixes it

Part 2, in Four Steps

One part today, in four steps — and the first step is yours.

2.1 · Exploration lab — find the settings that work.

2.2 · The 1-D example — the failure in one dimension.

2.3 · Signal propagation — the solution and its rule.

2.4 · The demo — verifying the result.

The exploration comes first; the explanation comes after it.

It Is 1985. Would You Have Made It Train?

You have written a neural network. Now you have to make it learn.



- The goal is modest — a loss that comes down, not a good model.
- Gradient descent has a convergence theory, so you have confidence.
- What you cannot be sure of is whether the guarantees apply here.

So you try settings, one after another, until one of them trains.

The Lab, and the Rules of the Game

One page in your browser, nothing to install. It trains on two spirals.

ruoyus.github.io/me2605/demos/first-trial-demo/

You may change — φ , seed, initialization, learning rate, momentum.

What is fixed in a seeting— depth and width, data, batch size, steps.

What you see — the training and test error — two curves, nothing else.

The rule — six scenarios, each with a target; clear all six.

Press Train, then start changing — and write down what you changed.

A Network That Will Not Train

Press Train. The error is 50% — as good as guessing.

- Several things can be changed.
- **Goal:** Find which one decides whether training works
- **If you already know the answer, please keep it to yourself.**

Expect to be stuck first.

Being stuck for 20 minutes will help you understand training better.

THE TASK

Six Scenarios — Clear Them All

Six scenarios, with different depth and the width.

Get the training error below the target.

S1 2 hidden layers, width 10 — training error below 15%

S2 4 hidden layers, width 10 — training error below 12%

S3 6 hidden layers, width 10 — training error below 12%

S4 4 hidden layers, width 16 — training error below 10%

S5 6 hidden layers, width 16 — training error below 12%

S6 8 hidden layers, width 16 — training error below 20%

Clear all six, then the page congratulates you.

Raise Your Hand — What Did You Find?

Time. Before we name anything, let us hear what you saw.

- Which settings got a scenario to train? Describe them to the class.
- What did the settings that failed have in common?
- What turns out to be the deciding factor?
- What changed when the network got deeper?

Energy Preserving Principle

The first principle to make training work: energy preserving.

Let $f(x)$ be the output of the neural net, the principle is:

$$\frac{f(x)}{x} = \Theta(1)$$

- $\Theta(1)$ here just means “not too large or too small” — not a rigorous claim.
- Not easy to analyze a general graph, so analyze FCN.

Does Gradient Descent Work Here?

We begin with the smallest possible problem.

$$F(w) = (w^7 - 1)^2$$

- Use gradient descent with a constant step size: $w \leftarrow w - \eta F'(w)$.
- Does it work —with one fixed η ?
- If not, what is the obstacle?

F is smooth, with a single minimum at $w = 1$ — and GD still fails from most initial points.

Try It: Pick an Initial Point and a Step Size

The same problem, running in your browser.

Two things are yours to choose, and nothing else changes.

ruoyus.github.io/me2605/demos/gd1d-lab/

What you choose — the starting point w_0 and the step size η .

What you see — where the iterates go and what F is worth. There is no picture of the function.

What counts as working — the iterates reach $w \approx 1$, so that F drops to about zero.

Gradient Explosion and Gradient Vanishing

Let's check the gradient at two typical points.

$$F'(w) = 2(w^7 - 1) \cdot 7w^6 = 14w^6(w^7 - 1),$$

$$F'(2) = 14 \cdot 2^6(2^7 - 1) = 14 \cdot 64 \cdot 127 = 1.1 \times 10^5,$$

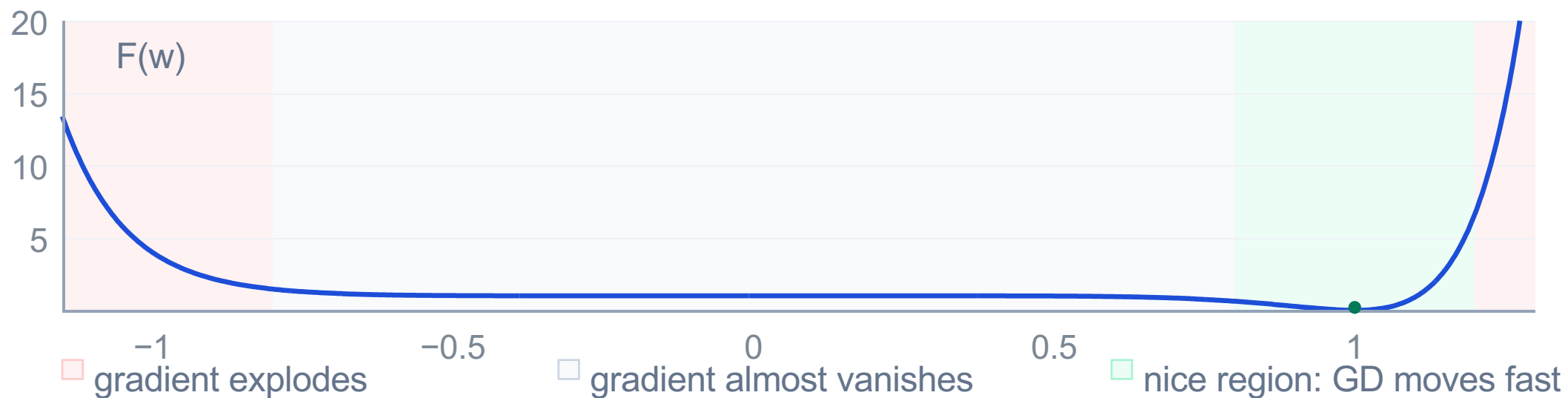
$$F'\left(\frac{1}{2}\right) = 14 \cdot \frac{1}{64} \left(\frac{1}{128} - 1\right) \approx -0.217.$$

Explosion — 1.1×10^5 at $w = 2$.

Vanishing — ≈ 0 near $w = 0$.

Where the Gradient Is Too Big and Too Small

Let's draw the image of the function.



Flat band: the gradient vanishes.

Steep walls: it explodes.

Near the minimum 1: the nice region.

Difficulty

How to resolve the issue of gradient explosion / vanishing?

$$\frac{\partial F_s}{\partial W_l} = e_l (z_{l-1})^\top$$

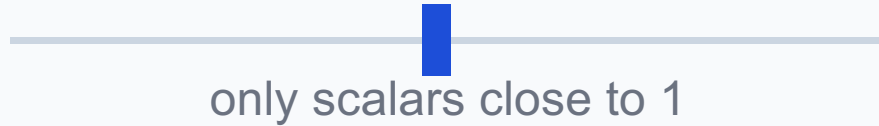
- The gradient scales with $\|z_{l-1}\|$ and $\|e_l\|$ — both signals must stay under control.

For a network, staying in the good region means preserving the energy layer to layer.

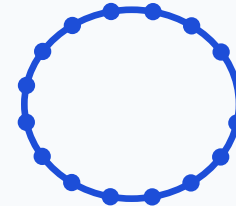
The “Good Region” Is Large in High Dimensions

In a high-dimensional space, the “good region” is “small” compared to the whole space, but large enough to find!

1-D: the good region is tiny

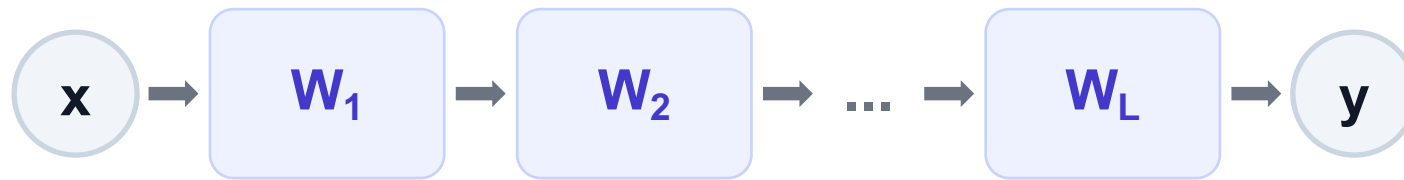


high-dim: good region is reasonably large



Linear Network: Simple Experiment

We start from a linear network of the form



we want $\frac{\|y\|}{\|x\|} = \Theta(1)$ for every input x

$$\frac{\|W_L W_{L-1} \cdots W_1 \cdot x\|}{\|x\|} = \Theta(1), \quad \forall x.$$

- For simplicity, we start by considering a single vector x .
- Experiment.

Script in MATLAB

```
clear;
d = 100;           % dimension for weight matrix W
L = 10;           % number of layers
x = ones(d,1);
norm0 = norm(x);
for i = 1 : L
    W = randn(d,d);
    x = W*x;
    ratio = norm(x)/norm0
end
```

**ratio is printed after
every layer — does it
stay near 1?**

What the Experiment Shows

Running the same script twice, with two different scalings of W , gives two very different pictures.

Naive choice ($W = \text{randn}(d,d)$) — leads to explosion or vanishing — the signal does not survive depth.

Tuned variance ($W \sim N(0, 1/d)$ per entry) — somewhat surprisingly, keeps the output signal near 1.

The variance of the weights is a control knob for signal propagation.

Two Initializations

Bottou–LeCun initialization

$$\mathbb{E}(W_{l,ij}) = 0, \quad \text{var}(W_{l,ij}) = \frac{1}{d_{l-1}}, \quad l = 1, 2, \dots, L; \forall i, j.$$

e.g. $W_{l,i,j} \sim \mathcal{N}(0, 1) \cdot \frac{1}{\sqrt{d}}$. Sometimes written as $\text{var}(W_{ij}) = \frac{1}{\text{fan-in}}$.

Xavier initialization

$$\mathbb{E}(W_{l,ij}) = 0, \quad \text{var}(W_{l,ij}) = \frac{2}{d_{l-1} + d_l}, \quad l = 1, 2, \dots, L; \forall i, j.$$

Sometimes written as $\text{var}(W_{ij}) = \frac{2}{\text{fan-in} + \text{fan-out}}$.

Expected Signal Strength

We first study one-layer signal propagation $y = W x$.

- With every $x_i = 1$, the expected squared output is 1.
- Claim 1 below makes this precise, and Corollary 1 extends it to multiple layers.

Claim 1: One-Layer Signal Propagation

Suppose the entries of W are independent with $\mathbb{E}[W_{ij}] = 0$ and $\text{Var}[W_{ij}] = 1/d_{\text{in}}$. Then:

- (i) $\mathbb{E} |\langle W_{i,:}, x \rangle|^2 = \frac{1}{d_{\text{in}}} \|x\|^2, \quad \mathbb{E} \left(\frac{1}{d_{\text{out}}} \|Wx\|^2 \right) = \frac{1}{d_{\text{in}}} \|x\|^2$
- (ii) if x is a random vector independent of W , the same holds with $\|x\|^2$ replaced by $\mathbb{E}\|x\|^2$.

part (i) holds for any fixed x ; part (ii) for x random and independent of W .

Why One Layer Works: the Proof

Proof of Claim 1:

$$\begin{aligned}\mathbb{E} \left| \langle W_{i,:}, x \rangle \right|^2 &= \sum_{j=1}^{d_{\text{in}}} \mathbb{E} [W_{ij}^2 x_j^2] + 2 \sum_{j < l} \mathbb{E} [W_{ij} W_{il}] \mathbb{E} [x_j x_l] \\ &= \sum_{j=1}^{d_{\text{in}}} \mathbb{E} [W_{ij}^2] \mathbb{E} [x_j^2] + 0 = \frac{1}{d_{\text{in}}} \mathbb{E} \|x\|^2.\end{aligned}$$

Independence kills the cross terms; $\text{Var}[W_{ij}] = 1/d_{\text{in}}$ provides the coefficient.

Read the assumptions off the proof: zero mean, special variance, and independence.

What the Derivation Actually Used

Three properties:

$E[W_{ij}] = 0$ — this is what makes every cross term vanish.

$\text{Var}[W_{ij}] = 1/d_{\text{in}}$ — the only other number in the answer.

Independence — of the entries from each other and from x .

Any distribution with these three properties work.

What Would Break It

The three properties are also a list of ways to fail. Take one away:

Drop the zero mean — the cross terms no longer cancel.

Wrong variance — the energy changes by a constant factor each layer.

— **exponential in depth.**

Dependent entries — the cross terms no longer cancel.

Gaussian, Uniform, or Binary?

The following distributions satisfy the desired properties, so Claim 1 holds.

Gaussian $N(0, 1/d)$ — yes — zero mean, right variance. Usual, not required.

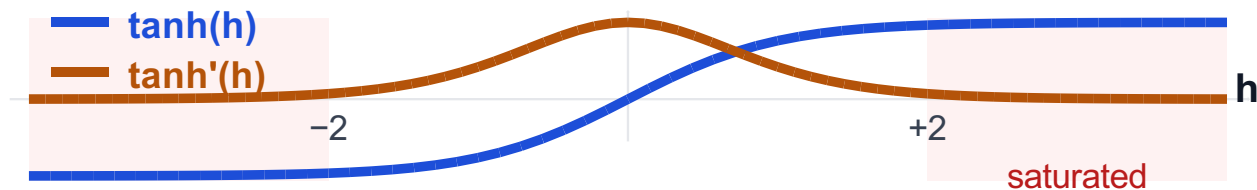
Uniform $U(-a, a)$ — yes — symmetric, so mean 0; $a^2/3 = 1/d$ gives $a = \sqrt{3/d}$.

Binary $\pm a$, half each — yes — symmetric, so mean 0; $a^2 = 1/d$ gives $a = 1/\sqrt{d}$.

Beyond linear nets: tanh activation

We discussed linear nets. Nonlinear activation?

Tanh activation function: may not work well



- **Note:** If h is large, then $\tanh(h)$ is large, then $\tanh(\tanh(h))$ is still close to $\tanh(h)$.
So the effects of tanh do not add up across layers.

ReLU activation

Tanh: derivative can be much smaller than 1.

ReLU activation: a newly discovered activation in around 2010.

$$\text{ReLU}'(h) = \mathbf{1}\{h > 0\} \in \{0, 1\},$$

$$\mathbb{E} \|\text{ReLU}'(h) \odot v\|^2 = \frac{1}{2} \|v\|^2 \quad (h \text{ symmetric, zero mean}),$$

$$\text{Var}(W) = \frac{2}{d_{\text{in}}} \quad \text{gives back the factor 2.}$$

What We Use Now: SiLU, GeLU, and the Gated Form



$$\text{SiLU}(t) = t \sigma(t) \quad (= \text{Swish}), \quad \text{GeLU}(t) = t \Phi(t).$$

$$\text{gated : } \text{SwiGLU}(x) = \text{SiLU}(xW) \odot xV.$$

σ is the logistic and Φ the Gaussian distribution's CDF function.

ReLU is not differentiable at 0. GeLU, SiLU are differentiable.

In addition, they do not map negative numbers to 0.

Compare: $\text{SiLU}(-1) = -0.269$, $\text{GeLU}(-1) = -0.159$. $\text{ReLU}(-1) = 0$.

Kaiming Initialization

Kaiming initialization (for ReLU)

$$\mathbb{E}(W_{l,ij}) = 0, \quad \text{var}(W_{l,ij}) = \frac{2}{d_{\text{in}}} \quad \text{or} \quad \frac{2}{d_{\text{out}}}.$$

Intuition

$$\mathbb{E}[\text{ReLU}(\xi)^2] = \frac{1}{2} \mathbb{E}[\xi^2]$$

for symmetric ξ : ReLU zeroes half the mass, so the second moment halves.

PyTorch Implementation

Xavier and Kaiming initialization:

```
torch.nn.init.xavier_uniform_(tensor, gain=1.0)
torch.nn.init.xavier_normal_(tensor, gain=1.0)
torch.nn.init.kaiming_uniform_(tensor, a=0,
mode='fan_in', nonlinearity='relu')
torch.nn.init.kaiming_normal_(tensor, a=0,
mode='fan_in', nonlinearity='relu')
```

- Default initialization for a linear layer and a conv layer in CNN:
Kaiming uniform, fan-in mode.

Does the Rule Actually Work?

We discussed theory.

Next, we use a demo website to see its impact on training.

```
ruoyus.github.io/me2605/demos/nn-signal-lab/
```

What you control — a gain g on the initialization scale, plus depth and width.

Set the gain to the theory prediction (Bottou-LeCun initialization, or Kaiming initialization).

What to look for — the norms of each layer's post-activation (signal strength) are all near 1.

How Today's Story Fits Together

- 1 **Backprop** — the gradient is a left pile and a right pile, both products of W .
- 2 **Exploration** — a real net will not train; you found which settings make it train.
- 3 **Key factor**— **gradient** explosion and vanishing, in a toy example.
- 4 **The principle** — keep $\|f(x)\|/\|x\| = \Theta(1)$.
- 5 **The theory** —setting i.i.d random initialization with certain variance can make the expected norm of each layer's output relatively stable.
- 6 **The check** —demo website that you see how initialization affects each layer's signal strength.

SUMMARY

Summary

What to Take Away — Backprop and Signal Propagation I

Back-propagation — the chain rule, one layer at a time: a gradient is finished when its error arrives.

The linearization trick — replace the nonlinearity by $D_l = \text{diag}(\phi'(h_l))$ — a product of matrices.

$\partial F / \partial W_l = e_l (z_{l-1})^T$ — at most rank 1 per sample, at most B per minibatch — postivativation times an error.

Energy-preserving principle — choose the variance of the weights so the expected signal keeps its size.

Classical initializations — Bottou–LeCun, Xavier and Kaiming initialization.